PHOTO RESTORATION
USING DIGITAL IMAGE PROCESSING

Automated scratch detection and localized image repair

Sai Chaithanya Gosula (BT24ECI057) · Sai Ram Ruttala (BT24ECI054)
Sai Tharun Geddam (BT24ECI003) · Siva Krishna Manchineella (BT24ECI053)

Indian Institute of Information Technology, Nagpur | Department of ECE

Abstract

Old photographs often contain scratches, cracks, and image noise. This project presents a web-based restoration workflow that detects candidate damage, builds a repair mask, and reconstructs masked regions using image inpainting.

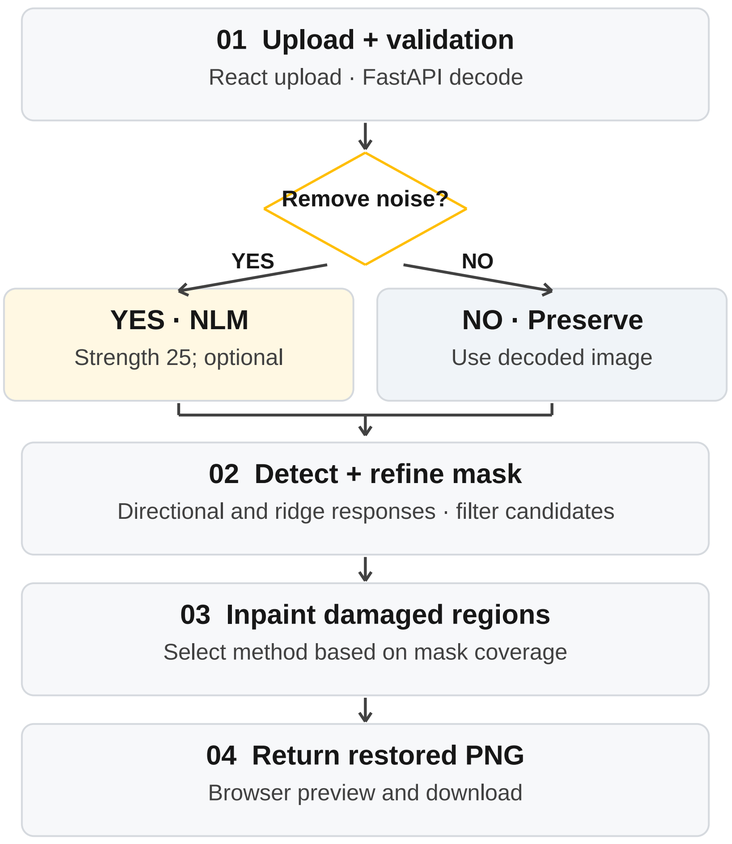
A React interface connects to a FastAPI backend. Noise removal is optional, allowing users to keep original detail when denoising is not needed.

Problem and Objectives

Physical photo damage can cross important image content, making manual repair slow and automatic restoration difficult.

- Detect bright, line-like scratches and cracks.
- Refine candidate pixels into a binary damage mask.
- Repair masked pixels while limiting changes elsewhere.
- Provide a simple upload, preview, and download workflow.

Processing Workflow



Technology Stack

- React frontend for image selection and result preview.
- FastAPI upload and PNG response endpoint.
- Python, OpenCV, NumPy, and Pillow for image processing.

Scope and Design Choices

The workflow targets visible, line-like scratches and cracks in a single uploaded photograph. A pixel-level damage mask limits where the repair stage changes the image.

Noise removal is optional because smoothing can suppress fine texture as well as unwanted noise. Leaving it off preserves the decoded image before damage analysis.

This is a classical image-processing workflow. It fills masked areas from surrounding pixels and does not infer missing scene details from a learned model.

Methodology

- Input images are decoded to 8-bit BGR arrays for OpenCV processing.
- Optional Non-Local Means (NLM) noise removal is off by default. When selected, it uses a conservative fixed strength.
- Crack detection combines directional brightness symmetry responses with a multi-scale ridge response.
- Mask refinement joins short gaps, filters candidate components by shape, thickness, and local contrast, then expands repair seams.
- Tharun's pipeline uses Navier–Stokes inpainting for very small masks and biharmonic inpainting for larger masks.

Backend and Frontend Integration

The browser sends a multipart image upload and an optional noise-removal flag to **POST /restore**.

FastAPI validates the file, calls the root processing pipeline, encodes the returned BGR image as PNG, and streams it back to the browser.

The React editor displays the original and restored images and provides a PNG download.

Experiments and Results

Test setup: qualitative inspection of an 800 × 500 portrait with a prominent vertical scratch at input, mask, overlay, and restored stages.

Stage	Method	Observed result
Noise option	NLM · strength 25	Off by default; both settings returned valid PNG output.
Mask · image02	Symmetry + ridge detection	6.94% coverage; scratch gaps and texture false positives remain.
Inpainting	Biharmonic	Repairs masked pixels; quality depends on mask accuracy.

Key Findings

Mask precision is the main quality bottleneck: texture creates false positives, while fragmented scratches are missed.

Optional denoising preserves detail when the input is already clean.

Interpreting the Mask

The 6.94% value is the share of pixels marked for repair, not a detection-accuracy score. Accuracy requires a manually labeled ground-truth mask for comparison.

The binary mask shows which pixels are sent to inpainting. Its overlay exposes false positives on ordinary portrait texture; unmasked scratch gaps explain damage retained in the output.

Judge the restored image together with the input and mask. A smooth-looking repair alone does not confirm that the correct regions were selected.

Conclusion

The project connects browser upload and preview to a modular restoration pipeline and demonstrates the path from input image to downloadable PNG.

Improving mask precision is the clearest next step because detection errors directly limit inpainting quality.

PROJECT LINK

PLACE QR CODE

Link to the project demo or repository

Implementation Results



Fig. 1 Input photograph



Fig. 2 Binary damage mask



Fig. 3 Detected damage overlay



Fig. 4 Output using the current mask

Limitations and Next Steps

Texture can be mistaken for damage, faint scratch segments can be missed, and classical inpainting cannot recover unseen details.

Next: compare masks with manual annotations, reconnect scratch fragments, reduce texture false positives, and add manual mask correction.

References

[1] A. Telea, “An Image Inpainting Technique Based on the Fast Marching Method,” Journal of Graphics Tools, 2004.

[2] M. Bertalmio et al., “Navier–Stokes, Fluid Dynamics, and Image and Video Inpainting,” CVPR, 2001.

[3] A. Buades et al., “A Non-Local Algorithm for Image Denoising,” CVPR, 2005.

Visual Result Notes

The overlay shows the detector also selecting facial and clothing texture around the prominent vertical scratch.

Missed mask pixels retain the original scratch; extra mask pixels can soften undamaged regions. This is why mask quality is central to the final result.